

Restaurant Review Sentiment Analysis

Project Report

Author	Kapil Mali
Domain	Natural Language Processing / Sentiment Analysis
Dataset	1,000 labeled restaurant reviews (binary sentiment)
Model	Feed-forward Neural Network (Keras) on TF-IDF features
Source	Restaurant_Rev.ipynb

1. Executive Summary

This report documents a sentiment classification system built on the Restaurant Reviews dataset (1,000 labeled reviews, binary sentiment). Reviews are cleaned, converted to TF-IDF features, and classified with a feed-forward neural network built in Keras (two hidden layers of 1000 and 500 units, with dropout regularization). The model was trained in two stages using early stopping, and evaluated on a held-out 10% test split.

Final result: the trained model reached **97.8% accuracy** on the training set and **77.0% accuracy** on the held-out 100-review test set, with a test F1-score of 0.81 for the negative class and 0.72 for the positive class. The gap between training and test performance indicates the model is overfitting the training data — expected given the network's size (1000+500 hidden units) relative to the dataset (900 training reviews).

2. Dataset

The dataset contains 1,000 restaurant reviews, each labeled 1 (liked) or 0 (not liked). The classes are evenly balanced (500 positive / 500 negative). Reviews are short, free-text sentences (e.g. *"Wow... Loved this place."*, *"Crust is not good."*), typical of informal customer feedback.

3. Methodology

3.1 Text Preprocessing

Each review is lowercased and stripped of non-alphabetic characters, then tokenized with NLTK's word tokenizer. Standard English stopwords are removed, with an explicit exception: the words **"not"**, **"no"**, **"bad"**, and **"never"** are kept in the stopwords-removal step, since these carry sentiment-relevant meaning that would otherwise be lost.

3.2 Feature Extraction

Cleaned review text is converted to numerical features using scikit-learn's **TfidfVectorizer** with default settings, producing a document-term matrix of 1,888 features across all 1,000 reviews. The vectorizer is fit on the **full dataset** before the train/test split is performed (see Section 5, Notes).

3.3 Train/Test Split

The TF-IDF matrix is split into training and test sets using scikit-learn's **train_test_split** with **test_size=0.1** and **random_state=40**, giving 900 training reviews and 100 test reviews.

3.4 Model Architecture

Layer	Configuration
Input	Shape = (1888,) — one input per TF-IDF feature
Dense (Hidden 1)	1000 units, ReLU activation
Dropout	Rate = 0.2
Dense (Hidden 2)	500 units, ReLU activation

Dropout	Rate = 0.5
Dense (Output)	1 unit, Sigmoid activation

The model is compiled with the **Adam** optimizer, **binary cross-entropy** loss, and tracks **accuracy** as its evaluation metric — a standard configuration for binary classification.

3.5 Training Procedure

Training was carried out in two stages on the same model, using Keras' **EarlyStopping** callback to halt training once validation performance stopped improving:

- **Stage 1:** EarlyStopping(patience=10), validation_split=0.1, batch_size=16, up to 50 epochs — stopped after **12 epochs**.
- **Stage 2:** EarlyStopping(patience=20), validation_split=0.2, batch_size=16, up to 50 epochs — continued training the same model for **23 further epochs**.

Because the second stage continues training the already-fitted model from Stage 1 (the model is not reset or rebuilt between the two fit() calls), the total training exposure is the combination of both stages rather than a single, independent run. See Section 5 for a note on how this affects interpretation of the learning curve below.

4. Results

4.1 Final Evaluation

Metric	Training Set (900 reviews)	Test Set (100 reviews)
Accuracy	97.78%	77.00%
Loss (Binary Cross-Entropy)	0.071	0.655

4.2 Classification Report (Test Set)

Class	Precision	Recall	F1-score	Support
0 (Negative)	0.81	0.80	0.81	60
1 (Positive)	0.71	0.72	0.72	40
Accuracy			0.77	100

The model performs somewhat better on negative reviews (F1 = 0.81) than positive reviews (F1 = 0.72) on this test split.

4.3 Learning Curve

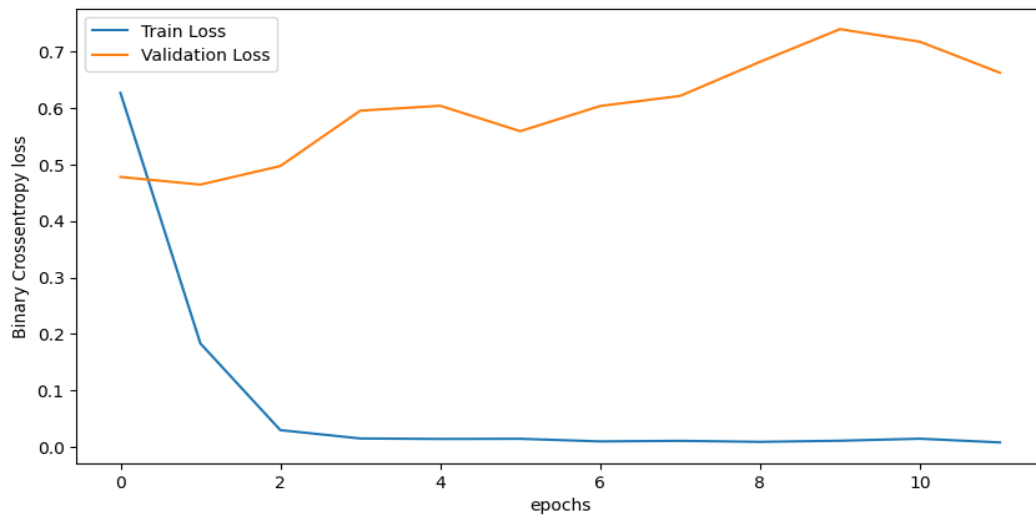


Figure 1: Training vs. validation loss, as generated in the notebook. The pattern is a textbook overfitting signature: training loss falls rapidly toward zero within the first few epochs, while validation loss flattens out and then begins climbing — meaning the model continues to fit the training data more closely without improving (and eventually while worsening) its generalization to unseen reviews.

4.4 Confusion Matrix (Test Set)

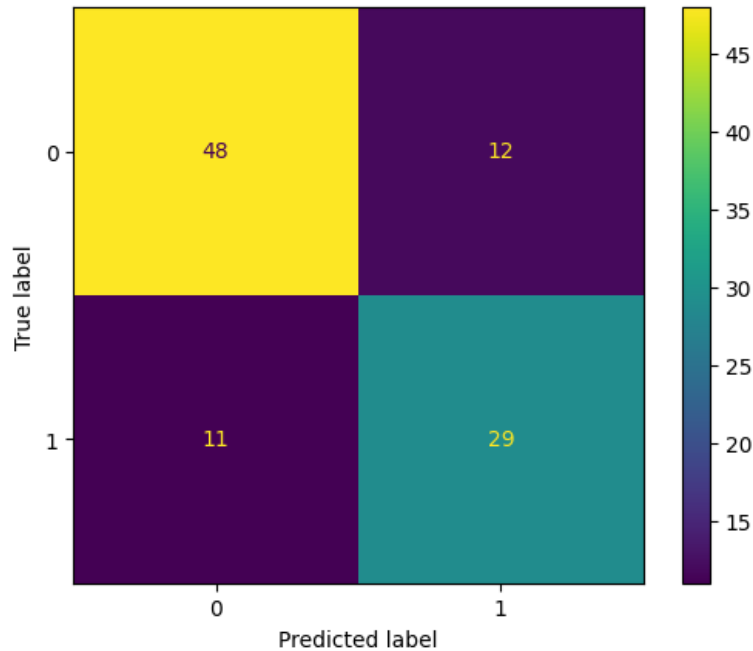


Figure 2: Confusion matrix on the 100-review test set. Of 60 true negative reviews, 48 were correctly classified (12 false positives). Of 40 true positive reviews, 29 were correctly classified (11 false negatives) — consistent with the classification report above.

4.5 Example Predictions

The notebook's `predict_sentiment()` function was tested on two reviews entered interactively:

Input Review	Predicted	Model Probability
"Wow... Loved this place."	Positive	0.99996
"bad"	Negative	0.0000054

Both examples — a clearly positive and a single strongly negative word — were classified correctly with high confidence, consistent with the model having learned strong, unambiguous sentiment cues well.

5. Notes on the Pipeline

Reviewing the notebook's code, two methodological details are worth flagging for awareness — they don't invalidate the results reported above, but they're useful context if these numbers are quoted elsewhere (e.g. in an interview):

TF-IDF fit before the train/test split. The vectorizer is fit on all 1,000 reviews before `train_test_split` is applied to the resulting matrix. This means the vocabulary and document-frequency statistics the model relies on were derived with the test reviews already included, which can modestly inflate test performance compared to fitting TF-IDF on the training split alone.

Two-stage training on the same model. The second `model.fit()` call (Stage 2, Section 3.5) continues training the model produced by Stage 1, rather than training a fresh model. The final metrics in Section 4 reflect the

model's state after both stages combined. This is a valid way to keep training longer, but it means the learning-curve chart and the final evaluation should be read as one continuous training run split across two cells, not two independently comparable experiments.

6. Conclusion

The notebook implements a complete, working sentiment classification pipeline: text cleaning with negation-aware stopword handling, TF-IDF feature extraction, and a regularized feed-forward neural network trained with early stopping. On its held-out test set, the model reaches 77% accuracy, with stronger performance on negative reviews than positive ones. The training/validation loss curve shows the model overfits after roughly 2-3 epochs, which is expected given the network's capacity (1000+500 hidden units) relative to the size of the training set (900 reviews after the split).

This report reflects the code and results contained in Restaurant_Rev.ipynb.